PR-AIE

Day-4 - Feb-27

Lec-12 - Data Types in python

→ Data Types:

The purpose of Data Types in Python is that "To allocate sufficient amount of main memory for storing the data".

→ We have 14 data types in python & classified into 6 categories.

I. Fundamental category Data Types

II. sequence " "

III. List " " [collections Data Type]

IV. Set " " ["]

V. Dict " " ["]

VI. None Type category Data Type.

I. Fundamental Data Types (4) [int, float, bool, Complex].

* 1. The purpose of Fundamental Category Data types is that "To store single value".

→ We have 4 Data Types in this category

1. int: is one of the pre-defined class.

a = 89

print (a, type(a))

89 <class 'int'>

The purpose of int Data Type is that "To store whole numbers or Integral values (values without Decimal places")

Page

Python Instruction — O/p

a = 10

print (a, type (a)) → 10 <class 'int'>

By using int data type, we can also store different types of Number systems [Decimal, Binary, octal, Hexa]

1. Decimal Number System: This s/s uses [0-9] & followed by human beings for their day-to-day operations.

* Default Number system; uses Digits 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Total Digits: 10

Base: 10

→ All base 10 values are called Decimal Number s/s.

→ Binary Number s/s: understandable by OS & processor.

→ Contains 2 digits; 0 & 1.

All base 2 literals are called Binary Number s/s values.

→ In Python, to store Binary data, the binary Data must be preceded with a letter 0b (or) 0B

Ex: a = 0b101

b = 0B010111

In Python, even though we store binary data, internally Python execution Environment implicitly converts into Decimal number System.

→ To convert decimal value into Binary value, use bin( ) which is a pre-defined function.

a = bin(18)

var-name = bin(Decimal/Octal/Hexadecimal)

→ bin(18) → 10010

Octal Number sys: understandable by micro processor units in 8086 or Assembly language programming.

→ uses 8 Digits; [0 - 7]

* Base; 8 → All base 8 values are called octal NS values

→ To store octal data, it must be preceded with a letter 0o (small o) or 0O (capital O)

zero zero

→ Note: To convert decimal/binary/hexadecimal into octal Number sys, we use a predefined function called oct( )

var_name = oct(Decimal/Binary/Hexadecimal)

oct(23) → '0027'

decimal octal

HexaDecimal NS: Uses 16 digits; [0 - 9] [A - F] (10 - 15) [a - f] ← we can use small also.

→ This NS is used in development of OS's.

→ This NS values are preceded with 0X.

→ All base 16 values are called HexaDecimal NS values.

hex( ) gives us value in HexaDecimal NS.

Lec-13 - Data Types - part 2

→ In Python, even though we store Hexadecimal NS, internally Python Execution Environment automatically (or) implicitly converts into Decimal NS.

a = '0XA'

print (a) → 10

→ hex(decimal / octal / binary) → gives value in Hexadecimal NS.

* **Float:** one of the pre-defined class.

→ a = 35.14 ; The purpose of float data type is "To store (float in python) Constant values" or floating point values (Numbers with decimal values).

Lec 14 - Data Types part 3

* → The float data type is also used for storing the floating point value in the scientific notation. The Syntax of scientific notation is

"mantissa e Exponent" → mantissa $\times 10^{\text{exponent}}$

↳ (can be +ve or -ve)

Ex: a = 3e5 ⇒ $a = 3 \times 10^5 = 300000.0$

Print (a, type(a))

300000.0, <'float'>

b = 3.24e5

print (b)

324000.0

↳ float ✓

c = 10e-2 ⇒ $10 \times 10^{-2} = 0.1$

↳ float

$5 \times 10^{-13} \Rightarrow 5e-13$

* a = 0.0000000000005 → To save the memory space, it is stored as a = 5e-13

* The Advantage of scientific Notation is to save memory space for storing Big floating point values.

Note: The float data type never allows the programmer to specify the binary, octal & Hexadecimal values. But it allows only Decimal number system

⇒ Ex. a = 0b111.0b101
↳ Error; invalid decimal literal.

→

bool: True / False → one of the Predefined 'class'

→ The purpose of 'bool' data type is "to store logical values (True or False)"

→ Here True and False are Keywords & considered as values for bool data type

→ Internally, True value is treated as 1.
False " " " as 0.

Print (True/True) → 1.0

Print (True/False) → ZeroDivision Error; division by zero

* On bool values, we can perform the Arithmetic operations also.

Complex Data type in python: One of the predefined class.

→ The purpose of complex datatype is that "To store complex values and perform operations on complex values".

→ General Notation; a+bj or a-bj

a → Real part b → Imaginary part

j represents sqrt(-1).

* a = 3+4j → Internally, Real parts and Imaginary parts are considered as float values.

Print(a.real)

3.0

→ * a = 10j ← valid ✓ : we can mention only ~~real~~ imaginary parts ✓

* c = -3j when -ve is present it gives

Print(c) → gives -0-3j

* To get Real & imaginary parts, we used pre-defined attributes

They are 1. real 2. imag

Lec-15 - String Datatype

String :→

The purpose of sequence category Data type is "to store sequence of values."

We have 4 types in sequence category Data Types

1. str 2. bytes 3. bytearray 4. range.

→ **Str :**

In application development, when we need to store the text data such as names, product information, user-friendly messages etc. we need strings to store & represent them.

In Python, we don't have Char data type.

we have str data type only.

using 'str' we can store both characters & strings.

→ Notation: ' ' or " " used for storing strings.

→ c = "K"

print(c, type(c))

K <class 'str'>

s1 = "Python3.14"

↳ String ✓

Ex: "Python@#312", "12345", 'K', -- are all strings. ✓

→ We can store sentences in str. also.

* We can store addresses also, using multi line of strings.

→ We use '''' '''' ← Triple double quotes to store multi lines of data. ✓ or Triple single quotes also fine here.

↳ works for single line of data also.

→ Memory Management of Str :-

→ Consider the str;

S = "PYTHON" ← str literal ✓

→ When this statement is executed, in the memory, the above str Data stored as follows.

(+ve indexing) (left to right)

	0	1	2	3	4	5
object S →	P	Y	T	H	O	N
	-6	-5	-4	-3	-2	-1

(-ve indexing)

Here also (left to Right)

→ Each character corresponds to some index as shown

→ Starts from 0 in forward; Default step; 1 (Positive indexing)

→ Starts from -1 in backwards; Default step; 1 (Negative indexing)

* Negative indexing also starts from forward direction only.

-6	-5	-4	-3	-2	-1
P	Y	T	H	O	N

* Positive indexing: first index; 0
; Last index; len(strobj) - 1

In Negative indexing; first index; - len(strobj)
last index; -1

* S = "Python PROG"

P	y	t	h	o	n		P	R	O	G

Operations on str data: We can perform 2 types of operations on strings.

1. Indexing operation :→ Purpose is to get one value from str object
2. Slicing operation;

* Simply, the process of obtaining one value from str object by passing valid index is called Indexing.

Syntax; S[0] strobj[Index]
S[-1]
S[-len(S)]

→ Here strobj is an object of <class, 'str'>
→ Index can be +ve or -ve.
→ If we enter valid index, then we get corresponding value from str object, else we get "IndexError" as result